

RED TEAM LEARNING PATH

MODULE 02

Networking Fundamentals for Red Teamers

Fokus modul

- Memahami bagaimana data bergerak dari aplikasi ke interface, lalu menjadi packet/frame di jaringan.
- Menguasai IPv4, subnetting, gateway, routing, NAT, TCP/UDP, ports, sockets, DNS, ICMP, dan dasar IPv6.
- Menggunakan iproute2, ss, dig, curl, tcpdump, Wireshark, netcat, Docker, dan Nmap pada target lokal/authorized.
- Membangun isolated Docker network untuk melihat hubungan subnet, route, port publishing, packet capture, dan service discovery.
- Membaca trafik dari perspektif Red Team dan Blue Team tanpa melakukan scanning atau interception pada sistem pihak lain.

Repository: github.com/VaxXinE/red-teaming

Versi 1.0 - 04 Oktober 2026

Cara Menggunakan Modul Ini

Networking adalah fondasi yang membuat enumeration, pivoting, Active Directory, web testing, dan lateral movement terasa masuk akal. Jangan menghafal port atau command tanpa memahami paket yang sedang bergerak. Untuk setiap lab: prediksi dulu apa yang akan terjadi, jalankan command, lalu cocokkan hasilnya dengan mental model lo.

0.1 Learning Objectives

- [] Menjelaskan hubungan application, transport, IP, link layer, dan interface.
- [] Membedakan MAC address, IP address, port, socket, route, gateway, dan DNS name.
- [] Menghitung network, broadcast, dan host range untuk subnet IPv4 umum.
- [] Menjelaskan perbedaan TCP dan UDP serta membaca TCP three-way handshake pada packet capture.
- [] Membaca interface, route, neighbor table, dan listening socket menggunakan iproute2/ss.
- [] Melakukan query DNS menggunakan dig serta membedakan record A, AAAA, CNAME, MX, NS, dan TXT.
- [] Menggunakan tcpdump/Wireshark untuk membaca trafik milik sendiri pada localhost atau isolated lab.
- [] Membuat Docker bridge lab dan melakukan service discovery terbatas terhadap container milik sendiri.
- [] Menyimpan PCAP dan network evidence secara aman serta tidak mem-publish data sensitif tanpa review.

0.2 Prerequisite

- Module 00 dan Module 01 selesai atau konsep scope, terminal, filesystem, process, dan permission sudah nyaman.
- Omarchy dapat menggunakan Docker dengan sudo.
- Gunakan jaringan sendiri. Semua scanning dalam modul diarahkan ke localhost atau Docker bridge pribadi.
- Sediakan sekitar 2-4 jam untuk teori dan 3-5 jam untuk hands-on, tergantung pengalaman.

0.3 Tools yang Dipakai

Tool	Peran di modul
ip / ss	Interface, address, route, neighbor, dan socket.
ping / traceroute	Reachability dan jalur paket.
dig	DNS query dan record inspection.
curl	Membuat HTTP connection yang mudah diamati.
nc (OpenBSD netcat)	TCP/UDP listener dan client lokal.
tcpdump	Packet capture dari terminal.
Wireshark	Analisis packet capture secara visual.
Docker	Membuat isolated network dan target lokal.
Nmap	Service discovery terbatas terhadap lab pribadi.

SECURITY NOTE

Packet capture dapat memuat cookie, token, hostname internal, IP privat, atau plaintext protocol. Simpan PCAP seperti evidence sensitif: permission ketat, jangan upload ke repository publik, dan hapus ketika tidak lagi diperlukan.

0.4 Instalasi Tool di Omarchy

iproute2 dan ping biasanya sudah hadir sebagai bagian dari base environment Arch. Tool tambahan yang dipakai di lab dapat dipasang dari official repositories:

```
omarchy pkg add tcpdump traceroute bind openbsd-netcat wireshark-qt nmap
```

Verifikasi command:

```
command -v ip ss ping traceroute dig nc tcpdump wireshark nmap
```

```
ip -Version  
nmap --version | head -n 1
```

WHY THIS MATTERS

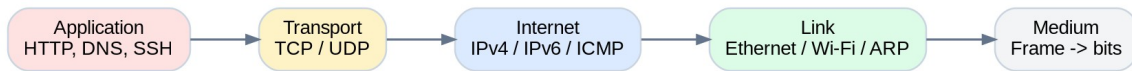
Arch sudah lama memprioritaskan iproute2. Gunakan ip/ss untuk materi modern; net-tools seperti ifconfig/netstat hanya perlu dikenali ketika membaca dokumentasi lama.

Daftar Isi

- 1. Mental Model Jaringan
- 2. OSI vs TCP/IP
- 3. Ethernet, MAC, dan ARP
- 4. IPv4 Addressing & Subnetting
- 5. Routing, Gateway, dan NAT
- 6. TCP, UDP, Ports, dan Sockets
- 7. TCP Connection Lifecycle
- 8. DNS Fundamentals
- 9. ICMP, Ping, dan Traceroute
- 10. IPv6 Fundamentals
- 11. Networking di Linux: iproute2 & ss
- 12. Packet Capture: tcpdump & Wireshark
- 13. Docker Networking untuk Lab
- 14. Service Discovery dengan Nmap
- 15. Firewall & Segmentation Concepts
- 16. Red Team vs Blue Team Perspective
- 17. Hands-on Labs
- 18. Mini Project: Network Baseline
- 19. Knowledge Check
- 20. Practical Challenge
- 21. Cheat Sheet
- 22. Referensi

1. Mental Model Jaringan untuk Red Team

Saat sebuah aplikasi mengirim data, data tidak langsung "pergi ke Internet". Operating system membungkus data melalui beberapa lapisan. Untuk troubleshooting dan pentesting, pertanyaan pertama bukan "tool apa yang dipakai?", tetapi "di lapisan mana masalah atau attack surface ini berada?"



Gambar 1 - Mental model: application data turun ke transport, IP, link, lalu media.

Objek	Pertanyaan praktis
DNS name	Nama ini resolve ke IP mana?
IP address	Host/network mana yang dituju?
Route	Lewat interface/gateway mana paket dikirim?
Port	Service/process mana yang menerima data?
Socket	Kombinasi protocol + address + port mana yang aktif?
Frame/MAC	Di local link, perangkat mana yang menerima frame?

THINK LIKE AN ATTACKER

Jika service tidak reachable, jangan langsung mencoba exploit. Cek berurutan: address -> route -> reachability -> port -> protocol -> application. Enumeration yang rapi mengurangi asumsi liar.

2. OSI vs TCP/IP

Model OSI berguna sebagai bahasa konseptual, sedangkan implementasi Internet sehari-hari lebih dekat ke model TCP/IP. Keduanya bukan daftar command, tetapi cara memetakan gejala.

OSI	TCP/IP praktis	Contoh
7 Application	Application	HTTP, DNS, SSH, TLS
6 Presentation	Application	Encoding, serialization, crypto representation
5 Session	Application	Session state
4 Transport	Transport	TCP, UDP
3 Network	Internet	IPv4, IPv6, ICMP
2 Data Link	Link	Ethernet, Wi-Fi, ARP/ND
1 Physical	Physical	Cable, radio, NIC signal

COMMON MISTAKE

Port bukan bagian dari IP address. Port adalah identifier pada transport layer (TCP/UDP). Satu host dapat memiliki banyak service karena setiap process dapat bind ke address/port berbeda.

3. Ethernet, MAC, dan ARP

Pada local Ethernet/Wi-Fi segment, IP packet dibawa di dalam link-layer frame. MAC address digunakan untuk pengiriman pada local link. Untuk IPv4, Address Resolution Protocol (ARP) memetakan IPv4 address ke link-layer address.

Konsep	Contoh	Catatan
MAC address	52:54:00:ab:cd:ef	Link-local identifier; bukan routing Internet.
ARP cache	192.168.1.1 -> aa:bb:cc:...	Hasil neighbor discovery IPv4.
Broadcast	ff:ff:ff:ff:ff:ff	Digunakan beberapa mekanisme local link.
Default gateway	192.168.1.1	MAC gateway yang dicari bila tujuan IP ada di network lain.

Inspect neighbor table pada host sendiri:

```
ip link
ip neigh
```

RED TEAM PERSPECTIVE

ARP/neighbor information membantu memahami host yang baru-baru ini berkomunikasi di local segment. Tetapi mengubah ARP behavior, spoofing, atau interception tidak dilakukan pada jaringan pihak lain tanpa explicit authorization.

4. IPv4 Addressing & Subnetting

IPv4 address panjangnya 32 bit. Prefix CIDR seperti /24 memberi tahu berapa bit yang menjadi network portion. Sisanya adalah host portion. Subnetting penting karena menentukan apakah target dianggap local atau harus dikirim melalui router.

4.1 Private IPv4 Ranges

Range	CIDR	Pemakaian umum
10.0.0.0 - 10.255.255.255	10.0.0.0/8	Enterprise/lab besar
172.16.0.0 - 172.31.255.255	172.16.0.0/12	Private networks/Docker/VPN
192.168.0.0 - 192.168.255.255	192.168.0.0/16	Home/small LAN

4.2 Cara Hitung Subnet

Untuk prefix /N, jumlah address = $2^{(32-N)}$. Untuk subnet klasik, network address adalah address pertama dan broadcast address adalah address terakhir. /31 dan /32 punya aturan khusus, jadi rumus "usable = total - 2" jangan dianggap universal.

Prefix	Total address	Usable klasik	Contoh mask
/24	256	254	255.255.255.0
/25	128	126	255.255.255.128
/26	64	62	255.255.255.192
/27	32	30	255.255.255.224
/28	16	14	255.255.255.240
/30	4	2	255.255.255.252

Contoh: 192.168.50.77/26. /26 berarti block size 64 pada octet terakhir. Range block: 0-63, 64-127, 128-191, 192-255. Karena 77 berada pada 64-127:

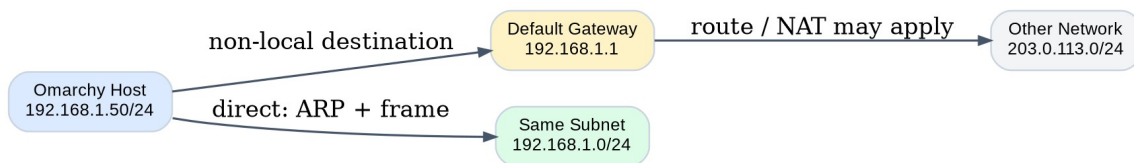
```
Network   : 192.168.50.64/26
First host: 192.168.50.65
Last host : 192.168.50.126
Broadcast : 192.168.50.127
```

PRACTICE WITHOUT CALCULATOR

Latihan subnetting manual sampai lo bisa cepat mengenali /24, /25, /26, /27, /28, dan /30. Setelah itu gunakan ipcalc/python sebagai verifikasi, bukan pengganti pemahaman.

5. Routing, Gateway, dan NAT

Routing menjawab pertanyaan: "untuk destination ini, packet keluar melalui interface mana dan next-hop apa?" Host membandingkan destination terhadap routing table dan memilih route paling spesifik (longest prefix match).



Gambar 2 - Local destination dapat dikirim langsung; network lain biasanya melalui default gateway.

```
ip route
ip route get 1.1.1.1
ip -6 route
```

NAT mengubah address/port pada boundary tertentu. Pada home router, source NAT sering membuat banyak private hosts berbagi satu public address. Pada Docker bridge, masquerading biasanya memberi container akses keluar melalui IP host.

IMPORTANT

NAT bukan firewall. Keduanya sering hadir bersamaan, tetapi fungsinya berbeda. Firewall memutuskan traffic mana yang diizinkan; NAT menerjemahkan address/port.

6. TCP, UDP, Ports, dan Sockets

TCP dan UDP sama-sama berjalan di atas IP, tetapi semantics-nya berbeda. TCP connection-oriented dan menjaga urutan/reliability. UDP connectionless dan lebih ringan; reliability bila diperlukan ditangani aplikasi/protocol di atasnya.

Aspek	TCP	UDP
Connection	Ya	Tidak ada handshake transport klasik
Reliability	Retransmission + ordering	Tidak dijamin transport
State	Stateful connection	Minimal transport state
Common examples	HTTP(S), SSH, SMB	DNS, DHCP, streaming/real-time use cases
Observation	SYN/SYN-ACK/ACK	Datagram request/response atau one-way

6.1 Port dan Socket

Port adalah angka 0-65535 pada TCP atau UDP. "Socket" dalam konteks observasi jaringan biasanya kombinasi protocol, local address:port, dan kadang remote address:port.

```
ss -ltn
ss -lun
ss -tnp
ss -uap
```

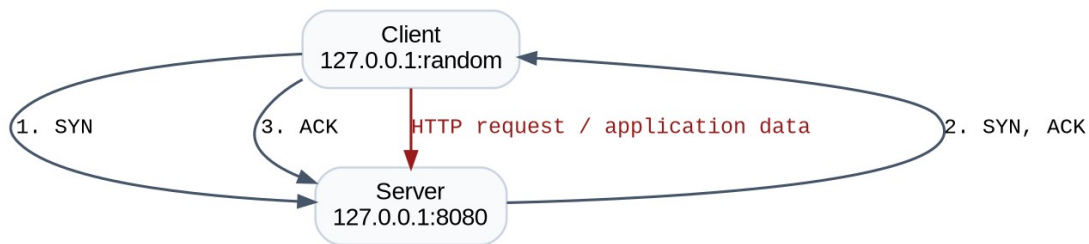
Binding ke 127.0.0.1 berarti service hanya menerima dari loopback host. Binding ke 0.0.0.0 berarti listen pada seluruh IPv4 interfaces yang tersedia. Dalam lab, prefer bind ke localhost ketika tidak perlu diakses dari LAN.

SECURE BY DEFAULT

Saat membuat server latihan, bind ke 127.0.0.1 atau isolated Docker network. Jangan gunakan 0.0.0.0 hanya karena lebih mudah, terutama pada laptop yang tersambung Wi-Fi publik/kantor.

7. TCP Connection Lifecycle

Three-way handshake menyinkronkan state awal kedua endpoint sebelum application data mengalir. Memahami flag SYN/ACK/RST/FIN membantu membedakan "port closed", "host unreachable", dan "application bermasalah".



Gambar 3 - TCP three-way handshake sebelum application data.

Flag	Makna singkat
SYN	Memulai/sinkronisasi connection.
ACK	Mengakui sequence/data.
FIN	Menutup connection secara teratur.
RST	Memutus/reset connection segera.
PSH	Mendorong data ke aplikasi; sering terlihat bersama ACK.

BLUE TEAM PERSPECTIVE

Banyak koneksi SYN ke banyak port dalam waktu singkat dapat terlihat seperti scanning. Bahkan pada lab, pahami bahwa packet pattern yang lo hasilkan adalah telemetry yang defender bisa lihat.

8. DNS Fundamentals

DNS memetakan nama ke data yang dibutuhkan aplikasi. Red Team memakai DNS untuk asset discovery dan memahami infrastructure; defender memantau query patterns, domain reputation, dan abnormal resolution behavior.

Record	Fungsi
A	Nama -> IPv4
AAAA	Nama -> IPv6
CNAME	Alias ke canonical name
MX	Mail exchanger
NS	Authoritative nameserver
TXT	Text metadata; sering SPF/DKIM/verification
PTR	Reverse mapping IP -> name

```

dig example.com A
dig example.com AAAA
dig example.com MX
  
```

```
dig example.com NS
dig +short example.com
```

Untuk melihat resolver host:

```
cat /etc/resolv.conf
resolvectl status 2>/dev/null || true
```

SCOPE NOTE

DNS query publik ke domain dokumentasi seperti example.com aman untuk latihan. Enumeration agresif/subdomain brute force terhadap organisasi nyata tetap mengikuti authorization dan scope engagement.

9. ICMP, Ping, dan Traceroute

ICMP membawa control/error messages untuk IP. ping biasanya memakai ICMP Echo Request/Reply untuk menguji reachability. traceroute mengandalkan TTL/Hop Limit dan respons dari router untuk memperkirakan jalur.

```
ping -c 4 127.0.0.1
traceroute 127.0.0.1
```

Tidak ada reply ping bukan bukti host mati. ICMP bisa diblok atau dibatasi. Selalu korelasikan dengan route, ARP/ND, TCP connection, atau application-level response.

10. IPv6 Fundamentals

IPv6 menggunakan 128-bit address. Modul ini hanya membangun fondasi agar lo tidak mengabaikan IPv6 ketika melakukan assessment. Banyak network modern dual-stack sehingga service bisa reachable lewat IPv6 walaupun IPv4 dibatasi.

Konsep	Contoh / catatan
Loopback	::1
Link-local	fe80::/10; hanya berlaku pada link/interface tertentu
Global unicast	Umumnya 2000::/3
Prefix LAN	Sering /64
Neighbor Discovery	Menggunakan ICMPv6; bukan ARP
Broadcast	IPv6 tidak menggunakan broadcast seperti IPv4; memakai multicast

```
ip -6 addr
ip -6 route
ping -6 -c 3 ::1
ss -ltn6
```

COMMON MISTAKE

Jangan menonaktifkan IPv6 hanya karena belum paham. Dari sisi security, blind spot IPv6 dapat membuat service exposure atau path yang tidak terlihat jika assessment hanya memeriksa IPv4.

11. Networking di Linux: iproute2 & ss

Di Arch/Omarchy, iproute2 adalah toolkit utama untuk interface, address, route, neighbor, dan traffic control. `ss` menggantikan banyak use case `netstat`.

Tujuan	Command
Ringkas interface	ip -brief address
Detail link	ip link
IPv4/IPv6 address	ip address / ip -6 address
Routing table	ip route / ip -6 route
Route lookup	ip route get <IP>
Neighbor cache	ip neigh
Listening TCP	ss -ltnp
Listening UDP	ss -lunp
Established TCP	ss -tnp

Workflow troubleshooting cepat:

```
ip -brief address
ip route
ip route get 1.1.1.1
ip neigh
ss -ltnu
```

MENTAL CHECK

Address menjawab "siapa gue?"; route menjawab "lewat mana?"; neighbor table menjawab "MAC siapa di link ini?"; socket menjawab "service apa yang sedang listen/terhubung?".

12. Packet Capture: tcpdump & Wireshark

Packet capture adalah microscope untuk jaringan. Gunakan capture untuk memvalidasi hipotesis: apakah SYN keluar? apakah DNS query dikirim? apakah server merespons? Jangan menjadikan capture sebagai aktivitas mass interception.

12.1 tcpdump

```
sudo tcpdump -i lo -nn
sudo tcpdump -i lo -nn 'icmp'
sudo tcpdump -i lo -nn 'tcp port 8080'
sudo tcpdump -i any -nn -c 20
```

Flag penting: `-i` interface, `-n/-nn` jangan resolve nama/service, `-c` batasi jumlah packet, `-w` simpan PCAP. Gunakan filter sempit supaya evidence mudah dianalisis.

12.2 Capture Filter vs Display Filter

Jenis	Kapan berlaku	Contoh
Capture filter	Sebelum packet disimpan; libpcap/BPF syntax	tcp port 8080
Display filter	Sesudah capture; hanya mengubah tampilan	tcp.port == 8080
Display filter	Filter SYN	tcp.flags.syn == 1
Display filter	Filter DNS	dns

IMPORTANT

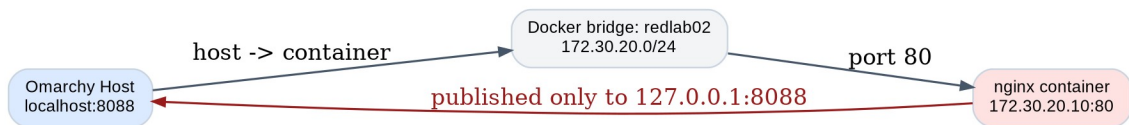
Wireshark capture filter dan display filter memakai bahasa yang berbeda. `tcp port 80` adalah capture filter; `tcp.port == 80` adalah display filter.

12.3 Workflow Aman dengan Wireshark

1. Capture traffic terkontrol dengan tcpdump pada localhost atau lab.
2. Simpan PCAP dengan permission 600.
3. Buka file PCAP di Wireshark sebagai user biasa.
4. Filter `tcp`, `dns`, `icmp`, atau `tcp.port == 8080`.
5. Pilih packet dan pelajari Frame -> Ethernet -> IP -> TCP/UDP -> Application.
6. Hapus PCAP bila berisi data yang tidak perlu dipertahankan.

13. Docker Networking untuk Lab

User-defined Docker bridge cocok untuk belajar karena container berada di subnet terisolasi pada satu host. Kita akan membuat subnet eksplisit agar perhitungan dan route mudah dipahami.



Gambar 4 - Topologi lab Module 02. Port host dipublish hanya ke localhost.

```

sudo docker network create \
  --driver bridge \
  --subnet 172.30.20.0/24 \
  redlab02

sudo docker run -d --rm \
  --name redlab02-web \
  --network redlab02 \
  --ip 172.30.20.10 \
  -p 127.0.0.1:8088:80 \
  nginx:alpine
  
```

Inspect network:

```

sudo docker network inspect redlab02
ip route | grep 172.30.20 || true
curl -I http://127.0.0.1:8088
  
```

WHY LOCALHOST PUBLISHING?

Docker `-p 8088:80` dapat publish ke host interfaces. Dengan `-p 127.0.0.1:8088:80`, lab web service hanya diekspos melalui loopback IPv4 host, mengurangi accidental exposure ke LAN.

14. Service Discovery dengan Nmap

Nmap masuk di sini hanya sebagai alat validasi konsep port/service. Enumeration mendalam akan dibahas lagi di Module 08. Pada Module 02, target harus localhost atau container `172.30.20.10` yang lo buat sendiri.

```

nmap -Pn -p 80 172.30.20.10
nmap -Pn -sV -p 80 172.30.20.10
nmap -p 8088 127.0.0.1
  
```

Baca hasil sebagai evidence: host -> port -> protocol -> state -> service -> version/response. Jangan langsung lompat dari "port open" ke "vulnerable".

LEGAL / SCOPE

Nmap adalah utility network discovery/security auditing. Scan hanya alamat yang termasuk scope. Pada modul ini, jangan mengganti IP lab dengan IP publik atau subnet kantor/kampus.

15. Firewall & Segmentation Concepts

Firewall memfilter traffic berdasarkan policy, sedangkan segmentation memisahkan asset/network agar compromise tidak otomatis membuka seluruh environment. Red Team perlu memahami firewall bukan sebagai "tembok ajaib", tetapi sebagai set rule yang berinteraksi dengan routes, address, ports, state, dan application flow.

Pertanyaan	Contoh observasi
Apakah route ada?	ip route get <target>
Apakah port listen?	ss -ltnu
Apakah packet keluar?	tcpdump pada interface
Apakah reply datang?	SYN-ACK/ICMP/TCP reset
Apakah application merespons?	curl / protocol-specific client
Apakah network segmented?	Target hanya reachable dari subnet/pivot tertentu

Read-only inspection bila nftables tersedia:

```
sudo nft list ruleset 2>/dev/null | less
```

DO NOT BREAK YOUR HOST

Jangan mengubah firewall utama laptop saat belajar Module 02. Kita fokus membaca state dan menggunakan Docker isolation. Rule manipulation akan lebih masuk akal setelah routing dan packet flow sudah kuat.

16. Red Team vs Blue Team Perspective

Aktivitas	Red Team melihat	Blue Team melihat
DNS query	Asset/service discovery	Resolver logs, unusual domains/volume
Port connection	Service availability	Firewall flow, SYN patterns, connection logs
HTTP request	Application behavior	Proxy/WAF/web access logs
Packet capture	Ground truth traffic	Biasanya dilakukan defender di controlled sensors
Nmap scan	Attack-surface mapping	Burst connections across ports/hosts
Pivot path	Reach internal-only asset	East-west traffic, unusual proxy/tunnel behavior

KEY IDEA

Setiap teknik ofensif meninggalkan observables. Belajar packet flow membuat lo lebih baik bukan hanya menyerang lab, tetapi juga memprediksi detection surface dan merancang engagement yang terukur.

17. Hands-on Labs

Semua lab di bawah berjalan pada host sendiri, localhost, atau isolated Docker bridge. Buat workspace terlebih dahulu:

```
LAB=~/.CyberSec/labs/module02
mkdir -p "$LAB"/{notes,evidence}
chmod 700 "$LAB" "$LAB/evidence"
cd "$LAB"
```

Lab 1 - Network Inventory

OBJECTIVE

Buat snapshot interface, address, route, neighbor, resolver, dan listening socket sebelum melakukan eksperimen lain.

```
ip -brief address | tee notes/01-address.txt
ip route | tee notes/02-route.txt
ip -6 route | tee notes/03-route-ipv6.txt
ip neigh | tee notes/04-neigh.txt
ss -ltnu | tee notes/05-listeners.txt
cat /etc/resolv.conf | tee notes/06-resolver.txt
```

Jawab di `notes/observations.md`: interface aktif apa? default route ke mana? service apa yang listen hanya localhost vs all interfaces?

Lab 2 - Subnetting Drill

Hitung manual sebelum menggunakan bantuan. Untuk tiap address, tulis network, broadcast, first/last host:

Address	Tugas
192.168.10.130/26	Temukan block /26.
10.10.8.17/28	Temukan block /28.
172.16.5.201/27	Temukan block /27.
192.168.1.10/30	Temukan dua usable host klasik.

Opsional verifikasi dengan Python stdlib:

```
python - <<'PY'
import ipaddress
for x in ['192.168.10.130/26', '10.10.8.17/28', '172.16.5.201/27', '192.168.1.10/30']:
    i=ipaddress.ip_interface(x)
    print(x, '->', i.network, 'broadcast=', i.network.broadcast_address)
PY
```

Lab 3 - Local TCP Listener & Socket

Terminal A:

```
nc -l 127.0.0.1 9001
```

Terminal B:

```
ss -ltnp | grep 9001
printf 'hello from client\n' | nc 127.0.0.1 9001
```

Perhatikan local address, port, state LISTEN, dan bagaimana data muncul di Terminal A. Ulangi listener jika connection selesai.

Lab 4 - TCP Handshake dengan tcpdump

Terminal A: jalankan HTTP server hanya pada loopback.

```
cd "$LAB"
python -m http.server 8080 --bind 127.0.0.1
```

Terminal B: capture traffic.

```
sudo tcpdump -i lo -nn 'tcp port 8080'
```

Terminal C:

```
curl -sS http://127.0.0.1:8080/ -o /dev/null
```

Cari SYN -> SYN,ACK -> ACK, lalu data dan teardown. Stop tcpdump dengan Ctrl+C.

Lab 5 - Simpan PCAP dan Buka di Wireshark

```
sudo tcpdump -i lo -nn -s 0 -c 30 \
'tcp port 8080' \
-w "$LAB/evidence/http-local.pcap"
```

Sambil capture berjalan, lakukan beberapa `curl` dari terminal lain. Setelah capture selesai, ubah ownership/permission:

```
sudo chown "$USER:$(id -gn)" "$LAB/evidence/http-local.pcap"
chmod 600 "$LAB/evidence/http-local.pcap"
wireshark "$LAB/evidence/http-local.pcap"
```

Gunakan display filter:

```
tcp.port == 8080
tcp.flags.syn == 1
http
```

PRIVACY CHECK

Sebelum commit repository, jalankan `git status` dan pastikan folder evidence/PCAP tidak ikut staged. Packet capture adalah data sensitif walaupun berasal dari lab.

Lab 6 - DNS Inspection

```
dig example.com A
dig example.com AAAA
dig example.com MX
dig example.com NS
dig +short example.com A
```

Catat perbedaan ANSWER, AUTHORITY, dan query time. Jika resolver memakai cache, ulangi query dan amati apakah behavior berubah.

Lab 7 - Build Isolated Docker Network

```
sudo docker network create \
--driver bridge \
--subnet 172.30.20.0/24 \
redlab02

sudo docker run -d --rm \
--name redlab02-web \
--network redlab02 \
--ip 172.30.20.10 \
-p 127.0.0.1:8088:80 \
nginx:alpine
```

```
sudo docker network inspect redlab02
curl -I http://127.0.0.1:8088
```

Observasi route `172.30.20.0/24` pada host, container IP, bridge interface, serta port publishing localhost.

Lab 8 - Local Service Discovery

```
nmap -Pn -p 80 172.30.20.10
nmap -Pn -sV -p 80 172.30.20.10
nmap -p 8088 127.0.0.1
```

Tulis hasil ke tabel `notes/service-map.md`: Target | Port | Protocol | State | Service | Evidence.

Lab 9 - Docker Packet Path

Terminal A:

```
sudo tcpdump -i any -nn 'host 172.30.20.10 and tcp port 80'
```

Terminal B:

```
curl -sS http://172.30.20.10/ -o /dev/null
```

Perhatikan interface yang terlihat dan source/destination address. Bandingkan dengan akses melalui `http://127.0.0.1:8088`.

Lab 10 - Cleanup & Evidence Review

```
sudo docker rm -f redlab02-web 2>/dev/null || true
sudo docker network rm redlab02 2>/dev/null || true
ss -ltnu | grep -E ':(8080|8088|9001)\b' || true
find "$LAB" -maxdepth 2 -type f -printf '%M %u:%g %p\n'
```

Stop Python HTTP server/netcat jika masih berjalan. Review notes dan pastikan evidence tidak world-readable.

18. Mini Project - Network Baseline Script

Buat script read-only yang mengumpulkan keadaan jaringan host tanpa scanning network. Script final juga disertakan dalam repo package modul ini.

```
#!/usr/bin/env bash
set -euo pipefail
umask 077

OUT_DIR="${1:-$HOME/CyberSec/labs/module02/evidence}"
mkdir -p "$OUT_DIR"
OUT="$OUT_DIR/network-baseline-$(date +%Y%m%d-%H%M%S).txt"

{
  echo '=== HOST ==='
  hostname
  echo
  echo '=== ADDRESSES ==='
  ip -brief address
  echo
  echo '=== ROUTES IPv4 ==='
  ip route
  echo
  echo '=== ROUTES IPv6 ==='
  ip -6 route
  echo
  echo '=== NEIGHBORS ==='
  ip neigh
  echo
  echo '=== LISTENERS ==='
  ss -ltnu
  echo
}
```

```

echo '=== RESOLVER ==='
if command -v resolvectl >/dev/null 2>&1; then
    resolvectl status
else
    cat /etc/resolv.conf
fi
} > "$OUT"

chmod 600 "$OUT"
printf 'Saved: %s\n' "$OUT"

```

Kriteria: tidak membutuhkan sudo, tidak melakukan active scan, tidak memanggil external "what is my IP" service, dan menghasilkan file 0600.

SECURITY REVIEW

Network baseline bisa berisi hostname, private IP, gateway, interface name, dan DNS server. Jangan commit output ke public repository. Commit script-nya, bukan hasil evidence-nya.

19. Knowledge Check

1. Apa beda IP address dan port?
2. Mengapa MAC address tidak dipakai sebagai Internet-wide routing address?
3. Apa fungsi ARP pada IPv4 local link?
4. Apa arti prefix /24?
5. Berapa total address pada /26?
6. Mengapa host 192.168.1.10/24 dapat mengirim langsung ke 192.168.1.20 tetapi biasanya memakai gateway untuk 10.0.0.5?
7. Apa beda route dan DNS?
8. Apa beda TCP dan UDP?
9. Sebutkan urutan TCP three-way handshake.
10. Apa arti service bind ke 127.0.0.1?
11. Apa risiko bind ke 0.0.0.0?
12. Apa beda `ss -ltn` dan `ss -lun`?
13. Apa beda capture filter dan Wireshark display filter?
14. Mengapa `ping` gagal tidak selalu berarti host mati?
15. Record DNS apa yang memetakan hostname ke IPv4?
16. Apa fungsi default gateway?
17. Apa beda NAT dan firewall?
18. Apa arti IPv6 `::1`?
19. Mengapa PCAP harus diperlakukan sebagai evidence sensitif?
20. Mengapa `nmap -sV` terhadap local lab belum otomatis membuktikan vulnerability?
21. Apa manfaat `ip route get <IP>`?
22. Apa hubungan Docker bridge subnet dengan routing table host?
23. Apa perbedaan port container 80 dan published host port 8088 pada lab?
24. Dari Blue Team perspective, apa observable dari port scanning?

20. Practical Challenge

SCENARIO

Lo menerima sebuah Omarchy host dan sebuah Docker container web lokal yang belum didokumentasikan. Tanpa melihat command dari lab sebelumnya, bangun network map dan buktikan packet flow-nya.

- [] Identifikasi interface aktif dan IPv4/IPv6 address.
- [] Temukan default route dan DNS resolver.
- [] Buat subnet `172.30.22.0/24` dan container nginx `172.30.22.20`.
- [] Publish port container 80 ke `127.0.0.1:8222` saja.
- [] Buktikan listener/port dari host menggunakan ss, curl, dan Nmap.
- [] Capture satu TCP handshake ke service dan tunjukkan SYN/SYN-ACK/ACK di Wireshark.
- [] Jelaskan route yang digunakan host untuk mencapai container.
- [] Buat `network-map.md` berisi target, address, route, port, service, dan evidence.
- [] Cleanup container/network dan verifikasi port sudah tidak listen.

Exit criteria: lo mampu menjelaskan "kenapa koneksi berhasil/gagal" berdasarkan packet flow, bukan hanya mencoba command acak.

21. Cheat Sheet

Tujuan	Command / filter
Interface ringkas	ip -brief address
Routing IPv4	ip route
Route lookup	ip route get <IP>
Neighbor IPv4	ip neigh
TCP listeners	ss -ltnp
UDP listeners	ss -lunp
Ping local	ping -c 4 127.0.0.1
DNS A	dig example.com A
Trace route	tracert <host>
Capture TCP port	sudo tcpdump -i <if> -nn "tcp port 8080"
Wireshark TCP	tcp
Wireshark SYN	tcp.flags.syn == 1
Wireshark DNS	dns
Docker networks	sudo docker network ls
Inspect Docker network	sudo docker network inspect <name>
Nmap one lab port	nmap -Pn -p 80 <lab-ip>

Subnet memory anchors: /24=256 total, /25=128, /26=64, /27=32, /28=16, /29=8, /30=4.

22. Referensi

Referensi utama diprioritaskan ke dokumentasi resmi dan RFC. Versi package Arch dapat berubah; command dan konsep tetap diverifikasi terhadap dokumentasi upstream.

ArchWiki - Network configuration / iproute2 - iproute2, interface, address, routing.

https://wiki.archlinux.org/title/Network_configuration

Arch Linux - iproute2 package - Package `ip`, `ss`, `bridge`, dan utilitas routing.

https://archlinux.org/packages/core/x86_64/iproute2/

Arch Linux - tcpdump package - Command-line packet analyzer.

https://archlinux.org/packages/extra/x86_64/tcpdump/

Arch Linux - Wireshark Qt package - Wireshark GUI package.

https://archlinux.org/packages/extra/x86_64/wireshark-qt/

Arch Linux - bind package - Menyediakan `dig` dan DNS tools.

https://archlinux.org/packages/extra/x86_64/bind/

Arch Linux - Nmap package - Network discovery/security auditing utility.

https://archlinux.org/packages/extra/x86_64/nmap/

Docker Docs - Bridge network driver - Isolation, subnet, masquerading, dan port publishing.

<https://docs.docker.com/engine/network/drivers/bridge/>

Docker CLI - docker network create - Membuat user-defined bridge network.

<https://docs.docker.com/reference/cli/docker/network/create/>

Wireshark User Guide - Capture, display filters, dan packet analysis.

https://www.wireshark.org/docs/wsug_html/

RFC 9293 - Transmission Control Protocol (TCP) - TCP specification.

<https://www.rfc-editor.org/rfc/rfc9293>

RFC 768 - User Datagram Protocol - UDP specification.

<https://www.rfc-editor.org/rfc/rfc768>

RFC 826 - Address Resolution Protocol - ARP.

<https://www.rfc-editor.org/rfc/rfc826>

RFC 1034 / 1035 - Domain Names - DNS concepts and implementation.

<https://www.rfc-editor.org/rfc/rfc1034>

RFC 4632 - Classless Inter-domain Routing (CIDR) - CIDR address strategy.

<https://www.rfc-editor.org/rfc/rfc4632>

RFC 8200 - IPv6 Specification - IPv6 core specification.

<https://www.rfc-editor.org/rfc/rfc8200>

Module 02 - Exit Criteria

- [] Gue bisa menjelaskan packet flow dari application sampai link layer dengan bahasa sendiri.
- [] Gue bisa menghitung subnet IPv4 umum tanpa bergantung penuh pada calculator.
- [] Gue bisa membaca interface, route, neighbor, dan listener pada Omarchy.
- [] Gue bisa membedakan TCP/UDP serta menunjukkan handshake TCP dari packet capture.
- [] Gue bisa membaca DNS record menggunakan dig.
- [] Gue bisa membuat isolated Docker bridge dan memetakan IP/port/service-nya.
- [] Gue bisa menggunakan Nmap hanya terhadap lab yang gue miliki dan menjelaskan hasilnya.
- [] Gue memperlakukan PCAP/network evidence sebagai data sensitif.

NEXT MODULE

Module 03 akan memakai fondasi ini untuk membedah HTTP request/response, headers, cookies, sessions, authentication, TLS, proxy, REST/JSON, dan traffic interception dengan Burp Suite.